

UNIDAD 8 · SEMANAS 13–14

Redes: sockets

📅 Semanas 13–14 · fechas por confirmar </> Laboratorio TD7 (resuelto + desafío IA)

~ 2 animaciones

MANUAL M8 · TEORÍA DE LA UNIDAD

Redes: sockets

Lectura base de la unidad 8 (≈ 1 h 30 min). Las salidas son reales: programas del TD7 ejecutados en el equipo del curso, sobre 127.0.0.1.

CONTENIDO

- | | | | |
|----|--------------------------------------|----|----------------------------|
| §1 | Un socket es un descriptor | §4 | Modelos de servidor |
| §2 | El ciclo de vida de una conexión TCP | §5 | Datagramas y la red de hoy |
| §3 | Direcciones, puertos y nombres | | |

§1 Un socket es un descriptor

Un **socket** es el extremo de una comunicación. `socket(domain, type, protocol)` devuelve un descriptor de archivo, como `open` o `pipe`: se usa con `read`, `write` y `close`, se hereda con `fork` y ocupa una entrada de la tabla de descriptores. El *dominio* dice qué direcciones usa: `AF_UNIX` (una ruta del sistema de archivos, en la misma máquina), `AF_INET` (IPv4) o `AF_INET6` (IPv6). El *tipo* dice cómo se comunica:

- **Conectado** (`SOCK_STREAM`, TCP en Internet): se establece una conexión y luego viaja un **flujo de bytes** confiable y ordenado, como por una tubería. Es el teléfono de la diapositiva 181.
- **No conectado** (`SOCK_DGRAM`, UDP): cada envío es un **datagrama** independiente, que puede perderse, duplicarse o llegar en otro orden. Es el telegrama.

La diferencia se mide. Tres `write` por TCP llegan en un solo `read` —TCP no sabe de mensajes—, mientras que por UDP cada `recvfrom` devuelve un datagrama entero; y si el receptor no lee a tiempo, UDP pierde datos sin que nadie reciba un error, incluso en la misma máquina:

Salida real

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom
#3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992
lost
kernel counter Udp RcvbufErrors grew by 9992
```

Consecuencia práctica: sobre TCP, la aplicación define dónde termina cada mensaje (HTTP/1 usa líneas y `Content-Length`; el servidor eco del TD7, el fin de línea). Y como con las tuberías, `read` y `write` pueden ser **parciales** (unidad 2): `write_all` repite hasta enviar todo.

Practica esto → A8.1, escenas 2 y 6 · TD7 · 4

§2 El ciclo de vida de una conexión TCP

El **servidor** crea un socket, le asigna dirección y puerto con `bind` y llama a `listen(fd, backlog)`: el socket se vuelve *pasivo* (estado LISTEN) y solo sirve para recibir conexiones. El **cliente** crea su socket y llama a `connect` con la dirección del servidor. Entonces los dos núcleos hacen el **saludo de tres vías**: SYN del cliente, SYN-ACK del servidor, ACK del cliente. Lo importante: **el saludo lo completa el núcleo**, no la aplicación. La conexión queda ESTABLISHED en la *cola de accept* del socket que escucha, y `accept` solo la saca de ahí y devuelve un **descriptor nuevo** para ese cliente (el de escucha sigue aceptando a otros).

El programa `tcp_states` juega los dos papeles en un proceso y le pregunta al núcleo el estado de cada socket con `getsockopt(TCP_INFO)`:

Salida real

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -          | server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED | server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED | server
ESTABLISHED
write/read                 | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2   | server
CLOSE_WAIT
server read                | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED      | server
CLOSED
                           | the client's end moved to a TIME_WAIT entry of the kernel

(60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      127.0.0.1:42772    127.0.0.1:PORT
```

El cierre es simétrico y en dos mitades. Quien llama primero a `close` (o `shutdown(fd, SHUT_WR)`) envía FIN y pasa por `FIN_WAIT1` y `FIN_WAIT2`; el otro recibe el FIN, queda en `CLOSE_WAIT` y su `read` devuelve 0 —fin de archivo, como en una tubería sin escritores—. Cuando este cierra a su vez (`LAST_ACK`) y llega el último ACK, el primero queda en **TIME_WAIT** durante 2·MSL (60 s en Linux), por si el último ACK se perdió y el FIN se repite. Esa entrada sobrevive al programa (la muestra `ss`) y es la razón de `SO_REUSEADDR`: sin él, reiniciar un servidor falla con `EADDRINUSE`.

El backlog no limita cuántos clientes atiende el servidor, sino cuántas conexiones ya establecidas pueden esperar a `accept`. Linux admite `backlog + 1`; con la cola llena descarta el SYN y el cliente lo reenvía con espera exponencial (1 s, 2 s, 4 s...):

Salida real

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```



Para curiosos: SYN cookies

Con un backlog muy chico (`./backlog 2 8`) el núcleo llega a responder con *SYN cookies*, una defensa contra la inundación de SYN que no guarda estado. Si el ACK final llega con la cola de accept llena, se pierde: el cliente cree estar conectado y el servidor no sabe de él. Los contadores de `nstat` lo muestran en el TD7 · 4. Por eso los servidores reales piden un backlog grande (el tope es `net.core.somaxconn`).

Practica esto → A8.1 ciclo de vida de una conexión · TD7 · 1 y 4

§3 Direcciones, puertos y nombres

Cada dominio tiene su estructura de dirección — `sockaddr_un` con una ruta de hasta 108 bytes, `sockaddr_in` con una dirección IPv4 y un puerto, `sockaddr_in6` para IPv6— y todas se pasan a `bind`, `connect` o `accept` como `struct sockaddr *` junto con su tamaño; `struct sockaddr_storage` alcanza para cualquiera. Los números dentro de la estructura van en **orden de red** (*big-endian*): por eso `htons` para el puerto y `htonl` para la dirección. El ejemplo de la diapositiva 187 lo olvidaba, y el error se puede ver:

Salida real

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:          the kernel bound port 22555
sin_port = htons(7000):   the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80):     bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110, sockaddr_storage
128
```

En un x86-64, 7000 = 0x1b58 se guarda como los bytes 58 1b; leídos como *big-endian* son 0x581b = 22555, y el servidor queda escuchando en otro puerto. La misma salida muestra otras tres cosas: `getaddrinfo` convierte un nombre y un servicio (de `/etc/hosts` y `/etc/services`, o del DNS) en una lista de direcciones IPv4 e IPv6 listas para usar —reemplaza a `gethostbyname`, que solo conocía IPv4, no era segura con hilos y salió de POSIX en 2008—; los puertos por debajo de 1024 exigen privilegios (`CAP_NET_BIND_SERVICE`); y los tamaños reales de las estructuras.

Un socket `AF_UNIX` no usa IP ni puertos: su dirección es un **archivo de tipo socket**. Además, el núcleo le dice al servidor quién está del otro lado (`SO_PEERCRED` : PID y UID), algo que systemd, D-Bus o `/var/run/docker.sock` usan para decidir qué permitir:

Salida real

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108
bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

Practica esto → TD7 · 4, partes 5 y 6

§4 Modelos de servidor

Un servidor pasa la mayor parte del tiempo **esperando**: a que llegue un cliente (`accept`) o a que ese cliente envíe algo (`read`). Cómo organiza esas esperas decide quién espera a quién:

- **Iterativo**: un solo hilo atiende a un cliente por vez. Si un cliente es lento, todos los demás esperan en la cola.
- **Un proceso por cliente** (`fork` tras `accept`): el hijo atiende y el padre vuelve a `accept` . Aislados entre sí, pero crear un proceso cuesta varias veces más que un hilo (medido en el TD6 · 5) y hay que evitar zombis.
- **Un hilo por cliente**: más barato y con memoria compartida, pero 10 000 conexiones serían 10 000 hilos con sus pilas.
- **Pool de hilos**: k hilos creados al inicio consumen conexiones de una cola acotada que llena el hilo principal —el productor—consumidor de la unidad 7—. Recursos acotados, pero k clientes lentos lo saturan.
- **Bucle de eventos**: un hilo pregunta al núcleo con `epoll_wait` qué sockets ya tienen datos y solo atiende a esos, así que nunca se bloquea por un cliente lento. Es la respuesta al problema C10K y el modelo de nginx o Node.js; `io_uring` (Linux 5.1) lleva la idea más lejos.

El TD7 implementa los cinco en `http_server.c` y los mide con 32 clientes, primero 1 y luego 8 que tardan 1 s en enviar su petición:

Salida real

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 982 ms, max 983 ms
```

Salida real

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 981 ms, max 981 ms
```

Salida real

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 1 ms, max 2 ms
```

Practica esto → A8.2 modelos de servidor · TD7 · 3 y 4

§5 Datagramas y la red de hoy

En modo no conectado no hay `listen` ni `accept`: el servidor hace `bind` a un puerto conocido y espera con `recvfrom`, que devuelve un datagrama y la dirección de quien lo envió; el cliente usa `sendto` con la dirección de destino (diapositivas 194–197). También se puede hacer `connect` sobre un socket UDP para fijar el destino y usar `send` / `recv`. UDP es la base de DNS, de la voz y el video en tiempo real y, desde 2021, de **QUIC** (RFC 9000), sobre el que va **HTTP/3** (RFC 9114): la confiabilidad y el cifrado se implementan encima, en el programa y no en el núcleo.

Dos cambios más respecto del curso original. Casi todo el tráfico va cifrado con **TLS** (https, 443): el socket transporta bytes cifrados y una biblioteca (OpenSSL, GnuTLS...) hace el trabajo. Y los servicios sin cifrar de la diapositiva 183 (`telnet`, `ftp`) se reemplazaron por `ssh` y https.

Practica esto → A8.1, escena 6 (UDP) · TD7 · 4